

Modyfikacje danych i fizyczne modele danych

Data changes and physical data models

dr hab. inż. Maciej Grzenda

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska

Wydział Matematyki i Nauk Informatycznych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Challenges related to placing data in databases

- How do we put data into databases?
- How do we update existing records?
- How do we remove records?
- How do we create, alter and drop tables in DBMS?

The answer: All these actions are also done with SQL statements

Objectives

- SQL is a key language in the databases domain
- SQL is used not only for querying the data, but also for:
 - Creating new records
 - Modifying existing records
 - Deleting records
- However, to put data into a database,
 - First, we need to develop a physical data model based on a logical data model
 - Next, the tables have to be created in a DBMS platform. This is also done with SQL statements
- Hence, during this lecture, we will discuss changes to both database content and table definitions

Data Manipulation Language (DML)

DML: key statements

- DML is a part of SQL used to manipulate the data
- DML statements are used for:
 - Creating new records (INSERT statement)
 - Modifying existing records (UPDATE statement)
 - Deleting records (DELETE statement)
 - Querying the data (SELECT statement)
- Less popular and partly vendor-dependant statements include:
 - MERGE – used to insert records and/or update records (the so called "upsert") in one statement
 - CALL (in Oracle DBMS, used to call a method)
 - BULK INSERT (in Ms SQL Server, used to import a datafile into a database)

Note that some sources treat SQL SELECT as not a part of DML

SQL INSERT statement

- Allows to insert a record or multiple records into a table and/or a view.
- This can be done in one of two ways:

```
1. INSERT INTO TableName  
   [ (Column_1, Column_2, ... ) ] VALUES  
   (Value_1, Value_2, ...)  
  
2. INSERT INTO TableName  
   [ (Column_1, Column_2, ... ) ]  
   SELECT ... FROM ...
```

In some SQL implementations (e.g. Ms SQL Server 2008+) multiple records can be inserted also by the INSERT statement in the first form. On other versions e.g. Oracle 10g, this method is not supported.

INSERT - discussion

- List of column names can be skipped. In that case value of each column must be provided
- Instead of VALUES phrase, a SELECT statement can be used. If so:
 - The number and types of output columns must correspond to the table definition and list of column names if used
 - Multiple records (as many as the number of rows in query result set) can be inserted by using a single INSERT statement

Sample INSERT statements

```
INSERT INTO Products VALUES  
(5000, 'Mineral water', 3, 5, '12  
bottles', 20.45, 45, 23, 10, 0)
```

In this case, all column values are defined

```
INSERT INTO Products  
(ProductId, ProductName , CategoryID)  
VALUES (5001, 'Mineral water', 5)
```

In this case, only some columns are supplied with data. Default values such as NULL will be attempted to be placed in all the remaining columns of the table.

```
INSERT INTO Customers  
(customerid, companyname) SELECT  
supplierid+10000, companyname from  
Suppliers
```

Finally, a result of SELECT statement is placed in the database. As in the previous case, default values are supposed to be placed in the columns not provided explicitly

SQL UPDATE statement

- Allows to modify existing records
- Can affect a single record or multiple records in a table
- In line with relational databases assumptions, a record is **not** modified based on "where" it is in a table. **There is no concept of "this" record** being updated
- To update exactly one record, a condition referring to the value of a primary key is needed
- Note that referential integrity may cause some UPDATE statements to fail

UPDATE statement

```
UPDATE TableName SET  
Column1=Value1[,Column2=Value2,...] [WHERE Condition ]
```

Examples:

```
UPDATE Customers  
SET City='Cracow'  
WHERE Country='Poland'
```

```
UPDATE Customers  
SET City= 'Cracow',Discount=10  
WHERE CustomerId=1045
```

```
UPDATE CUSTOMERS SET ... WHERE  
CustomerId IN (SELECT CustomerId from  
Orders WHERE ...)
```

Value can be defined through an expression e.g. $\text{NetValue} * 1.23$ or even an embedded SQL SELECT query

Update possibly many records matching condition

Update just one record based on its value of a primary key

The conditions defining which records to modify can be complex

SQL DELETE statement

- Allows to remove existing records
- Can remove a single record specified by using primary key value or multiple records in a table
- Similarly to UPDATE statements, the only way to refer to individual records, which are supposed to be removed, is by defining a condition referring to individual columns
- Note that referential integrity may cause some DELETE statements to fail

DELETE statement

```
DELETE FROM TableName [WHERE Condition]
```

Examples:

```
DELETE FROM Customers  
WHERE City='Warsaw'
```

Delete possibly many records
matching condition

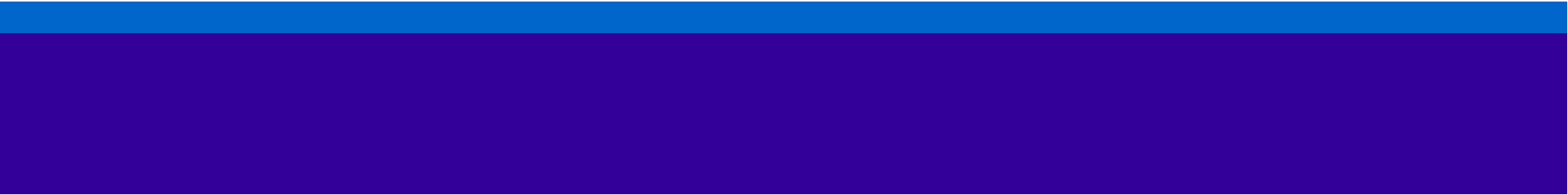
```
DELETE FROM Customers  
WHERE CustomerId=2045
```

Delete just one record based on
its value of a primary key

```
DELETE FROM Customers C  
WHERE NOT EXISTS (SELECT *  
FROM ORDERS O  
WHERE O.CustomerId=C.CustomerId)
```

The conditions defining which
records to delete can be complex

Differences in using aliases when referring to tables affected by UPDATE/DELETE statements are observed. While such aliases are not allowed in Ms SQL Server, they are supported by Oracle.



Data Definition Language and defining physical data models

Challenges related to creating tables

- Once we have logical data models, how do we develop physical data models based on them?
- How do we create tables?
- How do we create primary and foreign keys?
- Can we change table definitions?
- Should we create and modify tables programmatically or solely through a Graphical User Interface (GUI) of tools such as SQL Server Management Studio?

The answer: Creating physical data models for relational databases is quite straightforward, though physical and logical models are not identical. To create tables in an RDBMS, we will use SQL.

SQL – Data Definition Language

- Data definition language, also referred to as Data Description language (DDL) is an important part of SQL language devoted to the management of data types, structures, and constraints including:
 - Tables,
 - Constraints,
 - Indexes,
 - Databases and tablespaces,
 - Permissions,
 - ...

Defining tables in relational databases

- To develop tables in a relational database, we need to develop a physical data model
- This should be based on a logical data model.
- During this process:
 - **Entities** from logical data models are reflected in **tables** in physical data models,
 - **Attributes of entities** are reflected in **columns of the corresponding tables**
- The scale of adaptation of logical data models to develop relational, physical data models is limited.

The objective is to design a physical data model that will a) contain all data identified by the logical data model and b) maximise the performance of the database at the same time, e.g. by implementing changes in table definitions speeding up SQL queries.

Defining tables in relational databases

- When developing physical data models, we have to:
 - Select data types for columns out of these supported by DBMS of choice e.g. Oracle Database
 - Design indexes (to be discussed)
- We may also:
 - Design views, if needed
 - Decide about possible partitioning of tables, i.e.
 - Splitting a table into, e.g. two tables to have the most frequently used columns in one and the remaining ones in the other. This is called *vertical partitioning*
 - Splitting records into partitions. This is to avoid processing all rows when, most frequently, only the recent ones are queried. This is named *horizontal partitioning*.
 - Decide to add redundant columns containing values that could be calculated from other columns, but this would be too time-consuming.

Indexes and partitioning are applied to improve the performance of the queries. Is this something to discuss with business stakeholders and/or future users of database systems?

Developing physical data model (PDM) based on logical data model (LDM)

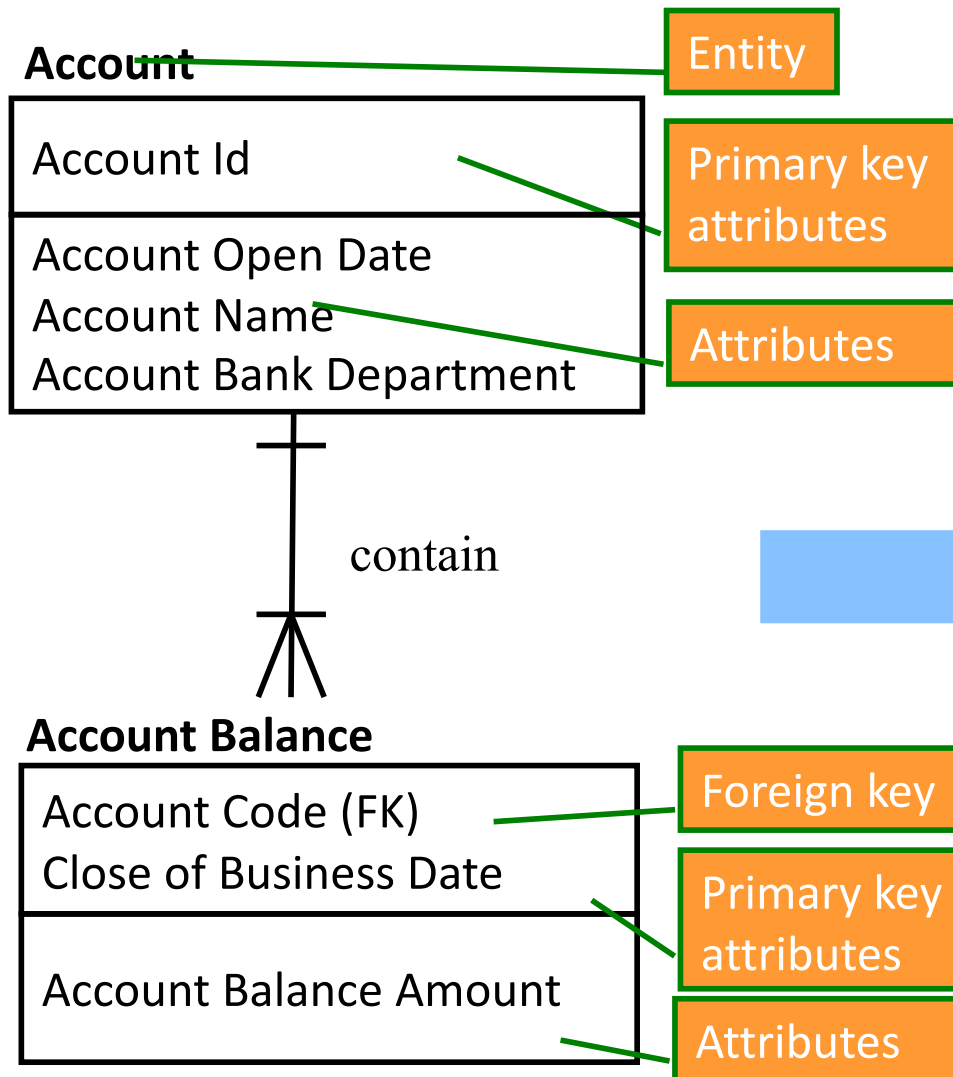


Fig. 1 Sample subset of financial LDM (based on [Hoberman, 2016])

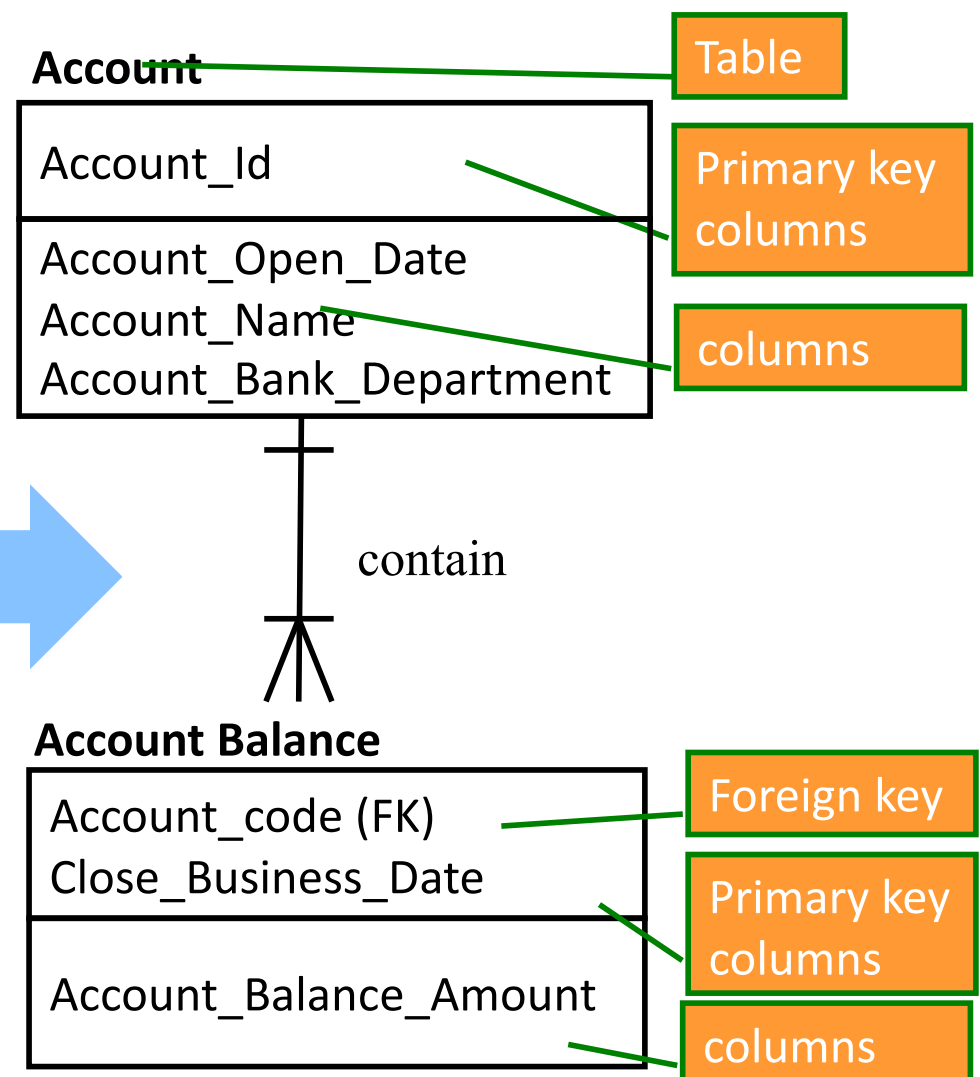


Fig. 2 Sample subset of financial PDM (based on [Hoberman, 2016]).
Column types skipped for clarity

Creating tables: CREATE TABLE statement

- **Syntax:**

```
CREATE TABLE TableName
(ColumnName ColumnType [Constraint] [,... ]
  [CONSTRAINT PKConstraintName PRIMARY KEY
  (
    Column1 [,... n]
  )]
  [CONSTRAINT FKConstraintName FOREIGN KEY
  (FKColumn1 [,... n])
  REFERENCES TableName (PKColumn1 [,... n])]
)
```

- **Example:**

```
– CREATE TABLE first (pname char(20) NOT NULL,
  psurname char(100) NOT NULL)
```

Only core CREATE TABLE syntax is shown. Numerous, both standard and non-standard options are offered by leading RDBMSs.

CREATE TABLE with constraints - example

Ms SQL Server

```
CREATE TABLE [dbo].[NewOrders]
(
    [OrderID] [int] NOT NULL,
    [CustomerID] [nchar](5) NULL,
    ...
    [ShipCountry] [nvarchar](15) ,
    [Discount] [decimal](6, 4) NULL
    CONSTRAINT [PKC_Orders] PRIMARY KEY
    (
        [OrderID]
    )

    CONSTRAINT [FKC_Orders_Customers]
    FOREIGN KEY([CustomerID])
    REFERENCES [dbo].[Customers]
    ([CustomerID])
)
```

Oracle Database

```
CREATE TABLE NewOrders
(
    OrderID int NOT NULL,
    CustomerID nchar (5) NULL,
    ShipCountry nvarchar2 (15) ,
    Discount decimal(6, 4) NULL,
    CONSTRAINT PKC_Orders PRIMARY
    KEY (OrderID),

    CONSTRAINT
    FKC_Orders_Customers FOREIGN
    KEY( CustomerID )
    REFERENCES Customers
    (CustomerID)
)
```

A table with a primary key and one foreign key is created.

More complex definitions - Oracle

```
CREATE
TABLE "SYS"."REG$"
(
  "SUBSCRIPTION_NAME" VARCHAR2(128 BYTE) NOT NULL ENABLE,
  "LOCATION_NAME"       VARCHAR2(256 BYTE) NOT NULL ENABLE,
  "USER#"              NUMBER NOT NULL ENABLE,
)
PCTFREE 10 PCTUSED 40 INITRANS 1 MAXTRANS 255 NOCOMPRESS LOGGING STORAGE
(
  INITIAL 65536 NEXT 1048576 MINEXTENTS 1 MAXEXTENTS 2147483645 PCTINCREASE 0
  FREELISTS 1 FREELIST GROUPS 1 BUFFER_POOL DEFAULT
)
TABLESPACE "SYSTEM" OPAQUE TYPE "ANY_CONTEXT" STORE AS LOB
(
  ENABLE STORAGE IN ROW CHUNK 8192 PCTVERSION 10 CACHE STORAGE(INITIAL 65536
  NEXT 1048576 MINEXTENTS 1 MAXEXTENTS 2147483645 PCTINCREASE 0 FREELISTS 1
  FREELIST GROUPS 1 BUFFER_POOL DEFAULT)
) ;
```

Note that some parts of the definition of this table were skipped. Advanced options are usu. not portable. In many cases they refer to data allocation settings. Such CREATE TABLE statements can be run without modifications on just one DBMS platform.

Modifying table definition: ALTER TABLE

- Allows to modify the structure of existing table
- Sample syntax:
 - ALTER TABLE TableName [ADD | ALTER COLUMN] ColumnName ColumnType
- Examples:
 - ALTER TABLE first ADD birthdate datetime
 - ALTER TABLE first ALTER COLUMN psurname varchar(200)

Not all table features can be modified. In some cases, a table has to be dropped and recreated with new settings, which may cause significant problems on production systems.

Removing a database object: DROP statement

- Allows to remove database objects of different types
- Syntax:
 - DROP TABLE | INDEX | PROCEDURE | ...
 ObjectName
 - ALTER TABLE TableName DROP COLUMN ColumnName
- Examples:
 - DROP TABLE first
 - ALTER TABLE first DROP COLUMN birthdate

DDL - comments

- DDL statements strongly depend on SQL implementation in the DBMS
- Different options are supported by DBMS vendors
- Refer to SQL documentation of the DBMS for details
- All DDL operations are subject to constraints e.g. it is not possible to remove a column that participates in a relation without removing the relation first

Further details on SQL dialects

DBMS	SQL reference
Ms SQL Server 2017	https://docs.microsoft.com/en-us/sql/t-sql/language-reference?view=sql-server-2017
Ms SQL Server 2019	https://docs.microsoft.com/en-us/sql/t-sql/language-reference?view=sql-server-ver15
Oracle 19c	https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/

Note that on some platforms, ANSI SQL standard compatibility can be to some extent controlled. As an example, ANSI_NULLS setting on Ms SQL Server makes it possible to decide how comparisons `ColumnName=NULL` are treated. When ANSI_NULLS is ON, rows with NULL values in `ColumnName` do not match the condition above, whereas ANSI_NULL=OFF changes this behaviour. Hence, the actual SQL dialect behaviour may depend also on compatibility settings.

DBMS platforms and databases

Revisiting the architecture and solutions

Why a database?

Flat files (TXT, XML, CSV etc.)	Database
Limited performance – difficult to find the data a user is looking for	Special structures (indexes) speed up information retrieval
Only simple queries can be easily answered	Any query can be answered (e.g. all the clients who bought product A last week, but have never ordered more than 50 items of product B)
Difficult to concurrently apply changes on behalf on many users	Concurrent updates are performed on the fly. State-of-the-art transactional processing is involved to ensure data consistency
Limited data security – anyone can damage/modify data files	Complex permission system securing the data
Difficult to make a backup of a file that is constantly modified	Various backup strategies incl. incremental backups

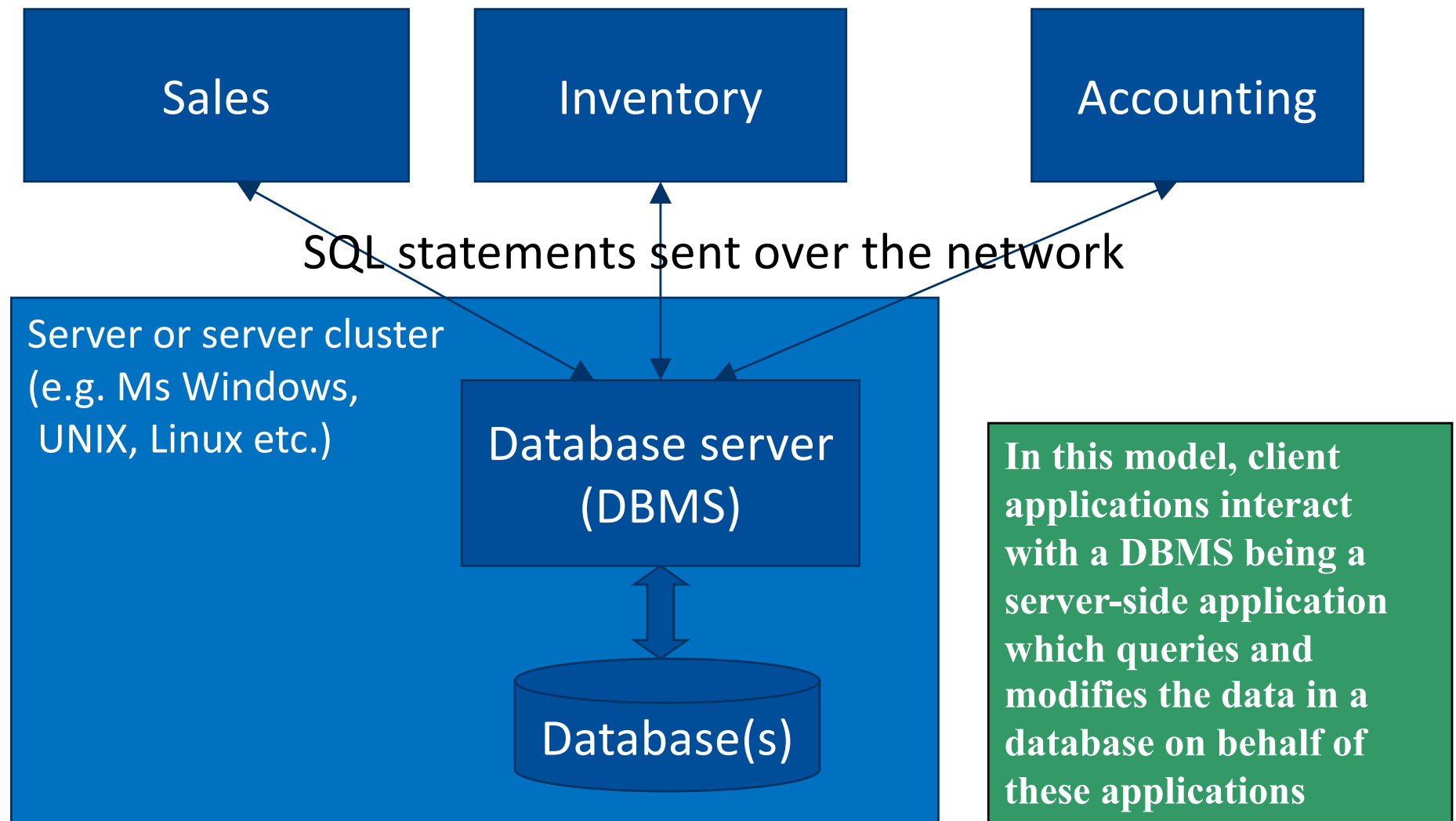
Most applications need to store some data permanently i.e. not only in in-memory variables, but on the disk. **Whenever multiple users share the data** (warehouse system, sales application, internet bookshop, social portals, banking system, insurance systems, ...) **a database is needed!**

DBMS - categories

- DBMS can be classified by:
 - The way data is managed:
 - a DBMS engine embedded in a database application or
 - a DBMS server i.e. server application that works on behalf of database applications and is accessible over the network
 - The way data is treated:
 - Relational databases oriented on records,
 - Object databases oriented on ... objects
 - Document databases i.e. databases storing data as documents e.g. JSON documents

Since databases based on relational approach (also referred to as relational databases) are most frequently used, unless otherwise stated, we will focus on relational databases.

Client-server approach: sample solution



Client-server approach

- Applications access databases by communicating with a DBMS server-side service, thus:
 - Only results of queries are transferred over the network,
 - Improved scalability of the solution is observed when databases get larger due to:
 - Limited increase in network communication,
 - Increased demand for computing resources (CPU, memory) occurring on the server only since data processing occurs mostly on the server
 - The security of the data can be improved, as:
 - Client application can not access data files
 - DBMS can enforce security constraints incl. detailed permission system at a table/column level,
 - DBMS can ensure data integrity.
 - Large databases with the volume of data of terabytes or even petabytes size (1PB=1000TB) can be handled

Client-server DBMS platforms

	Sample platforms and versions
Sample platforms (ranked based on https://db-engines.com/en/ranking)	1. Oracle 9i, 10g, 11g, 12c, 18c, 19c, 21c, 23ai,... 2. MySQL 3. Microsoft SQL Server 2000, 2008, 2014, 2016, 2019, 2022 ... 4. PostgreSQL 5. MongoDB 6. IBM DB2
The no. of concurrent users	ca. 1-5000-millions-... users
Benefits	Commercial relational platforms offer the most advanced mechanisms to a) maintain data quality, b) make data secure and c) enable high availability. This includes transactional processing, complex permission systems, complex backup strategies, stand-by servers, audit options, ...

Note that top 4 DBMS platforms are relational platforms. 7 out of 10 most popular DBMS platforms of db-engines ranking are based on relational approach. All top platforms rely on a client-server approach.

Summary

- SQL is used for querying data, but also for data modifications and managing various items in a database, such as tables, views, stored procedures, user accounts and more
- Even before SQL statements can be used to create tables and populate them with data, a physical data model has to be created
- Based on the physical data model, a database is created using the DBMS platform selected for the system

Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html





**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.